

CSCE 735 PARALLEL COMPUTING · MODULE 12 · LECTURE NOTES

Patterns, Frameworks, and Portability

Writing it once and running it anywhere.

MODULE OUTCOMES

- Apply the standard parallel patterns as a design vocabulary.
- Compare the major portability frameworks.
- Explain what performance portability means and costs.
- Choose a framework from the constraints.
- Explain why parallel code ages badly.

1 Patterns as a design vocabulary

MOST PARALLEL CODE IS A COMPOSITION OF SIX PATTERNS

Map, reduce, scan, stencil, gather/scatter, and partition (Module 06 §3) account for the overwhelming majority of parallel computation across every domain in this program — rendering, simulation, databases, and image processing alike.

Recognising which pattern a computation is gives you the parallel algorithm, its work and depth, the known pitfalls, and usually a tested library implementation — all without further analysis.

And it supplies a vocabulary for design discussion that is considerably more precise than 'parallelise the loop'. 'This is a segmented scan followed by a scatter' communicates the structure, the cost, and the likely bottleneck in one sentence.

The engineering is in the composition. A fused map-reduce never materialises the intermediate array, so it makes one pass over memory instead of two — which for a bandwidth-bound kernel (Module 07) is the entire optimisation, and halves the runtime.

Composition	Why it beats the parts separately
map → reduce, fused	No intermediate array is written or read. One pass over memory rather than three. For a bandwidth-bound computation this is the dominant factor.
map → scan → scatter	Stream compaction — the standard parallel filter (Module 11). Produces a densely packed output with no atomics and no contention.

Composition	Why it beats the parts separately
stencil → stencil (temporal blocking)	Fuse several timesteps over a spatial block , computing redundant halo values in exchange for keeping the block in cache across all of them. Trades extra computation for locality , which is the right trade on bandwidth-bound hardware — and it is underused.
scan → segmented scan	Operate on many independent sequences in one pass, using flags to mark segment boundaries. Essential for sparse and irregular data.
reduce → broadcast	An allreduce (Module 10) — and recognising it as such means using the optimised collective rather than two separate operations.

2 Portability frameworks

Framework	Targets	Character
OpenMP	CPU; GPU via target offload.	Pragma-based and incremental — you can parallelise one loop without restructuring. Ubiquitous and supported by every major compiler. The GPU offload support is improving and is not yet competitive with native.
Intel TBB	CPU.	C++ task-based with work stealing (Module 06). Excellent composability — nested parallelism does not oversubscribe.
Kokkos	CPU, CUDA, HIP, SYCL.	C++ abstractions over execution and memory spaces , with architecture-dependent data layouts. The de facto standard in US national laboratories , which is why it is well supported.
SYCL	CPU, GPU, FPGA.	Single-source C++ and an open Khronos standard. Intel's oneAPI is the most prominent implementation.
CUDA	NVIDIA only.	The most mature tooling, libraries, and documentation, and the least portable. A genuine trade rather than an oversight.
std::execution / C++ parallel algorithms	CPU; some GPU.	Standard C++ with no extra dependency. Still maturing, and the implementation quality varies considerably.

Kokkos and SYCL exist because laboratories were forced into them. Having procured machines with NVIDIA, AMD, and Intel GPUs in successive generations, and holding application codes with decades of accumulated physics, they could not rewrite for each. **The institutional pressure is why these frameworks are serious rather than academic.**

PORTABLE IS NOT PERFORMANCE-PORTABLE

Portable code compiles and runs on several architectures. That is the straightforward part, and several frameworks achieve it.

Performance-portable code achieves a good fraction of each machine's attainable peak — which is substantially harder, because the architectures want structurally different things.

A GPU wants coalesced access across threads, thousands of resident threads, and no divergence. A CPU wants cache blocking, vectorisable inner loops, and a thread count matching the core count. These are not merely different tunings; **sometimes the optimal data layouts directly conflict.**

So the frameworks' real contribution is abstraction over layout. A Kokkos View selects row-major or column-major ordering according to the target architecture, from identical source. **That is the actual engineering** — the parallel-for wrapper is the easy part.

THE LAYOUT CONFLICT, CONCRETELY

On a GPU, consecutive threads must access consecutive addresses to coalesce (Module 08 §2). If thread i handles column i of a matrix, the array must be stored so that moving between columns moves one element in memory.

On a CPU, each thread wants a contiguous block so that it stays in its own cache lines, avoids false sharing (Module 02), and vectorises over its inner loop. That requires the opposite ordering.

These are opposite memory layouts for the same logical array, and no single hard-coded choice serves both. A program written for one runs correctly and badly on the other.

A framework that parameterises the layout resolves this at compile time per target, with the index expressions generated accordingly. **Declaring the array yourself as a raw pointer does not**, and this is the most concrete answer to 'why not just use OpenMP and CUDA separately'.

3 Why parallel code ages badly

- **Core counts rise.** A granularity that balanced well across 8 cores is too coarse for 128 (Module 06), and a chunk size tuned for one is wrong for the other.
- **The ratio of compute to memory bandwidth rises.** Compute capacity has grown faster than bandwidth for decades, so the roofline's ridge point moves right — and **a kernel that was compute-bound becomes bandwidth-bound** without a line changing (Module 07). The correct optimisation strategy inverts.
- **Vector widths grow.** Hand-written SSE intrinsics do not use AVX-512, and code written with them is frozen at the width it was written for — which is the strongest argument for letting the compiler vectorise (Module 02).
- **New functional units appear.** Tensor cores, matrix engines, and specialised accelerators cannot be used by existing code at all, and reaching them generally means a library call or a rewrite.
- **And the tuned constants were tuned for a machine that no longer exists.** Tile sizes, chunk sizes, thread counts, unroll factors, and block dimensions were all measured on specific hardware, and all of them are now wrong. **This is the most actionable item**, because it is the easiest to design for.

WRITE FOR RETUNING RATHER THAN FOR ONE MACHINE

Parameterise the tuning constants. Tile size, chunk size, thread count, unroll factor, and block dimension should be named constants or runtime parameters — never literals scattered through the kernel. **Retuning then becomes changing values rather than finding them.**

Keep a correctness reference implementation that is obviously right and makes no attempt to be fast. Every optimisation is then checkable, and the reference does not rot because it is never optimised. CSCE 647 and 649 required this for the same reason.

Keep the benchmark with the code, in the repository, runnable with one command. **Retuning on new hardware is then a script, not a project**, and it is far likelier to actually happen.

And prefer libraries for the hot primitives, because somebody else retunes them for every architecture generation. **This is the argument of Modules 05 and 09 restated**, and it is the single most effective defence against ageing.

AUTOTUNING IS THE SYSTEMATIC VERSION OF THIS

Rather than choosing tuning constants by hand, search the parameter space automatically on the target machine, measuring each candidate and keeping the best.

This is how ATLAS, FFTW, and cuBLAS achieve their performance. FFTW measures many transform decompositions at first use and caches the plan; ATLAS generates and benchmarks hundreds of matrix multiply variants at install time. **The library is fast on your machine because it measured your machine.**

The search space is large — the product of several parameters, each with many values — so the search methods matter: pruning with analytical models, hill climbing, and increasingly learned cost models that predict performance without running the candidate.

And it is the honest conclusion of this course. The machine changes faster than anyone can retune by hand, so the right engineering response is to build the retuning into the software rather than to tune harder.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
McCool, Robison & Reinders — Structured Parallel Programming	The pattern catalogue and composition material of §1.
Kokkos documentation and tutorials (free)	The performance-portability abstractions of §2, including the layout handling that is the real contribution.
SYCL specification and oneAPI documentation (free)	The open-standard alternative.
Frigo & Johnson — The Design and Implementation of FFTW3 (free)	Autotuning done properly , by the people who did it. The planner description in §3 is here in detail.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- Express five computations from earlier courses in this program as compositions of the six patterns.
- Implement a map followed by a reduce as two passes and as a fused operation. Measure the difference and relate it to the roofline.
- Implement temporal blocking for a stencil** and measure the trade between redundant computation and improved locality.

- 4 Write the same kernel in OpenMP and in CUDA. Count the lines that differ and classify each difference.
- 5 Write it once in Kokkos and build for both targets. Compare performance against each native version and report the fraction achieved.
- 6 **Demonstrate the layout conflict:** write a kernel with the GPU-optimal layout and measure it on a CPU, and vice versa.
- 7 Take a kernel with hard-coded tile and block sizes and parameterise them. Sweep the parameters on your hardware and find the best.
- 8 Write an autotuning script that searches that space automatically and caches the result.
- 9 Find a piece of parallel code more than five years old and identify which of §3's five changes have invalidated its assumptions.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the six patterns and say what recognising one gives you.
- 2 Give five compositions and say why fusion is the most valuable.
- 3 Compare six portability frameworks on targets and character.
- 4 Distinguish portable from performance-portable.
- 5 Describe the layout conflict between CPU and GPU concretely.
- 6 Give five reasons parallel code ages.
- 7 Give four practices that make retuning feasible.
- 8 What is autotuning, which libraries use it, and why is it the honest conclusion?